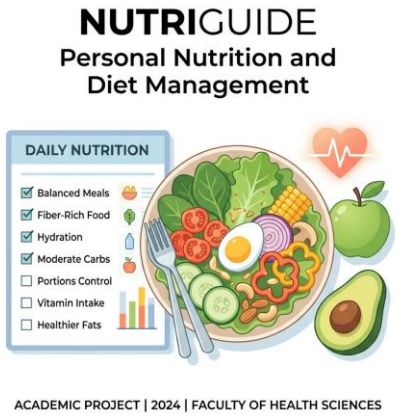
॥ वसुधैव कुटुम्बकम् ॥

A DBMS MINI PROJECT REPORT ON

# NutriGuide — Your Personal Nutrition Companion




**Batch: B3 | Group: GRP-1**

**Submitted by:**

**Tejas Jagdish Patil (26070122510)**

**Veer Parmar (26070122508)**

**Postiwala Vatsal Dipakbhai (26070122509)**

**Parth Bhayani (26070122512)**

*Under the Guidance of*

**Dr. Balkrishna Patil**

Department of Computer Science

Symbiosis Institute of Technology (SIT), Pune

Symbiosis International (Deemed University)

Academic Year 2026–2027

# INDEX


| Sr. No. | Title | Page No. |
|---|---|---|
| 1. | **Introduction** | 2 |
| 2. | **Problem Statement (Step 1)** | 3 |
| 3. | **System Architecture and Modules** | 4 |
| 4. | **Functional and Non-Functional Requirements (Step 2)** | 5–6 |
| 5. | **Entities, Relationships and Attributes (Step 3)** | 7–8 |
| 6. | **ER and EER Diagrams (Step 4)** | 9–10 |
| 7. | **Relational Schema (Step 5)** | 11 |
| 8. | **Anomalies (Step 6)** | 12 |
| 9. | **Functional Dependencies (Step 7)** | 13–14 |
| 10. | **Normalization (Step 8: 1NF to BCNF)** | 15–17 |
| 11. | **Implementation of Tables in MySQL (Step 9)** | 18–22 |
| 12. | **Functions, Procedures, Triggers and Queries (Step 10)** | 23–30 |
| 13. | **Application of Codd's 12 Rules** | 31–32 |
| 14. | **Individual Contributions (Step 11)** | 33 |
| 15. | **Conclusion** | 34 |
| 16. | **References** | 35 |
| **App. A** | **Complete SQL Script (NutriGuide.sql)** | 35–36 |
| **App. B** | **Stored Functions, Procedures, Views & Triggers** | 37–38 |


## 1. Introduction

Tracking daily nutrition is surprisingly difficult in everyday student and working life. Most of us eat meals on erratic schedules, grab quick snacks between classes or meetings, and rarely have an accurate idea of our actual calorie and macronutrient intake. When students or gym-goers attempt to fix their diet, they typically write food names in spiral notebooks, smartphone notes apps, or basic Excel sheets. In practice, this manual record-keeping quickly falls apart. Scaling protein and calorie numbers by hand every time portion sizes change is tedious. Food names get entered with spelling inconsistencies, and data ends up scattered across months without any clear structure or summary.

To solve this practical problem, our group designed and built NutriGuide, a desktop database application powered by a Java Swing user interface, an embedded Java HTTP REST backend running on port 8081, and a relational MySQL 8.0 database engine using InnoDB. Developed as our Database Management Systems (DBMS) mini-project, our main goal

was to take database concepts taught in the classroom and apply them directly to a working software system. We focused on solid relational schema design: eliminating insertion, update, and deletion anomalies through normalization up to Boyce-Codd Normal Form (BCNF), preserving data integrity with explicit foreign key rules (CASCADE and RESTRICT), and using stored database programs to run heavy aggregations directly inside the database engine.

Our database schema consists of ten connected tables that manage the complete nutrition workflow:

**1.** User Profiles and Health Metrics: Storing basic demographic information (age, gender), physical measurements (height in cm, weight in kg), and physical activity levels along with securely hashed user credentials.

**2.** Nutrition Goals: Storing daily targets for calories, protein, carbohydrates, and fats. We implemented active-flag versioning so users can update targets over time without erasing their previous goal history.

**3.** Food Item Catalog: A verified catalog of food items organized into primary dietary categories (Grains, Proteins, Dairy, Fruits, Vegetables), complete with reference calories and macronutrient densities per standard 100g serving.

**4.** Multi-Item Meal Logs: Real-time logging of breakfast, lunch, dinner, and snack events, broken down cleanly into individual food line items using the associative junction table meal_item.

**5.** Diet Planning: Allowing users to set up weekly or monthly meal schedules in advance with defined start and end dates.

**6.** Dietary Recommendations: Automated guidelines that compare user goals against real consumed intake to identify caloric surpluses, deficits, and protein shortfalls.

**7.** Database Audit Logging: An automated audit trail written by database triggers on meal insertion, ensuring full accountability without requiring extra lines of application code.

The backend implementation relies on MySQL 8.0 features including primary keys, candidate key uniqueness (UNIQUE), foreign keys with explicit cascade and restrict policies, and declarative CHECK constraints to ensure invalid data never enters the tables. Above the basic tables, we implemented stored functions (dynamic BMI calculation), stored procedures with output parameters for daily summaries, database views for instant aggregations, and triggers for automated validation and auditing.

## 2. Problem Statement (Step 1)

When people try to record and track their nutrition by hand or through casual spreadsheets, they repeatedly run into practical data management hurdles:

**1.** Scattered and Disorganized Records: People tend to note daily meals in one place, body weight measurements in another, and their fitness goals on random sheets of paper. Because there is no central database linking these pieces together, analyzing changes in body weight against actual caloric intake over time becomes nearly impossible.

**2.** Repetitive Calculation Errors: Calculating daily totals of calories, proteins, carbs, and fats requires multiplying portion sizes by nutrient percentages for every single item on a plate. Performing these calculations manually every day leads to frequent arithmetic errors, giving users a false impression of what they actually consumed.

**3.** Data Redundancy in Flat Files: When food intake is saved in simple spreadsheets, common food items (like Boiled Eggs or Cooked White Rice) get their full nutrient facts re-typed for every logged meal. If someone discovers an error in the nutrient values and updates one row, all historical rows still hold the old value, creating conflicting and untrustworthy records.

**4.** No Input Validation: Spreadsheets and text notes do not stop users from typing invalid or nonsensical numbers, such as negative weights, zero heights, or negative portion sizes.

**5.** Lack of Transaction Atomicity: In reality, a meal consists of several food items eaten together. If an application crashes halfway through saving a multi-item meal, a manual or poorly designed system leaves orphaned records where a meal is recorded without its food items.

**6.** Missing Audit History: Traditional spreadsheets do not track when a record was logged, updated, or removed, making it impossible to audit past changes or identify user activity history.

**7.** Disconnected Goal Comparison: Without relational queries linking logged meals to active nutritional targets, users cannot easily know whether they hit their daily caloric surplus or deficit.

### Proposed Solution

NutriGuide directly resolves these flaws by implementing a centralized, fully normalized relational database in MySQL. The relational design eliminates redundant data, enforces strict primary and foreign key rules, validates inputs at the storage engine level with CHECK constraints, automatically computes daily metrics using stored procedures and views, and safeguards meal logging through atomic database transactions.

## 3. System Architecture and Modules

### 3.1 Three-Tier Relational Topology

To keep the application modular, maintainable, and easy to debug, we separated NutriGuide into three distinct architectural tiers:

**1.** Presentation Layer (Java Swing Client): A desktop interface built using standard Java Swing components. It contains dedicated windows for user workflows (LoginFrame, RegisterFrame, DashboardFrame, MealLogFrame, NutritionGoalFrame, RecommendationFrame, ReportFrame, and ProfileFrame). To make sure the GUI never freezes while waiting for network or database responses, all HTTP requests run asynchronously in the background using SwingWorker threads, keeping the Event Dispatch Thread (EDT) responsive.

**2.** Application & Logic Layer (Java HttpServer): A lightweight REST server running locally on http://127.0.0.1:8081. It handles route dispatching, input validation, and business logic across service classes (AuthenticationService, NutritionService, MealService, FoodService, RecommendationService, ReportService, and UserService). It manages user sessions through token authentication and protects user passwords using salted PBKDF2-HMAC-SHA256 with 600,000 hash iterations before communicating with MySQL over JDBC.

**3.** Relational Database Layer (MySQL 8.x Engine): The core storage tier runs on MySQL 8.x using the InnoDB engine. It provides ACID transactional safety, enforces relational integrity (CASCADE / RESTRICT), executes stored functions and procedures, maintains aggregation views, and automatically invokes audit triggers.

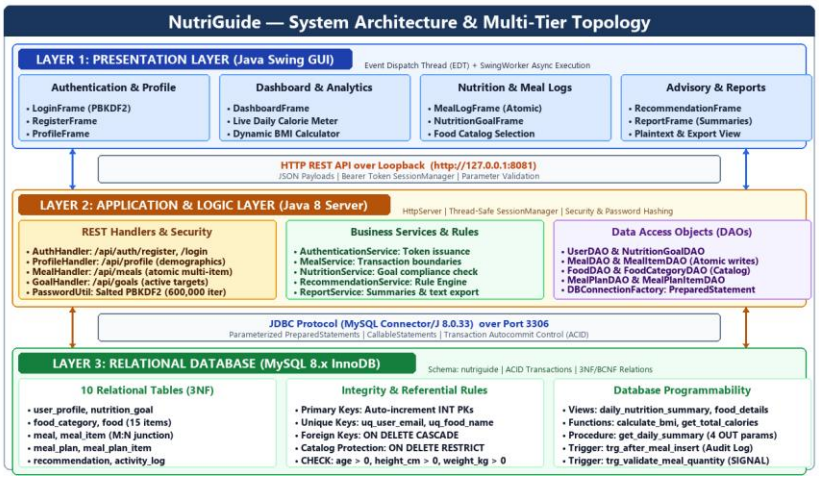



*Figure 3.1: NutriGuide System Architecture & Multi-Tier Topology*

### 3.2 System Modules

**1.** User Profile & Authentication Module (user_profile):

- Handles account registration, login verification, and demographic tracking.
- Stores user height, weight, age, gender, and physical activity level.
- Keeps passwords secure using salted PBKDF2-HMAC-SHA256 hashing so plaintext passwords are never saved.

**2.** Nutrition Goal Management Module (nutrition_goal):

- Lets users configure target daily calories, protein, carbohydrate, and fat amounts.
- Supports historical tracking by marking older goals inactive (is_active = FALSE) whenever a new goal is set.
- Enforces the business rule that each user has exactly one active goal at any given time.

**3.** Food Catalog & Categorization Module (food_category, food):

- Stores a standardized catalog of foods organized into primary groups (Grains, Proteins, Dairy, Fruits, Vegetables).
- Keeps track of serving sizes in grams along with calories and macronutrient densities.

**4.** Meal Logging & Transaction Module (meal, meal_item):

- Records breakfast, lunch, dinner, and snack entries for authenticated users.
- Uses the associative junction table meal_item to link meals with multiple foods and their consumed portion multipliers.
- Executes meal insertion inside atomic database transactions (START TRANSACTION / COMMIT).

**5.** Meal Planning Module (meal_plan, meal_plan_item):

- Enables users to prepare upcoming diet schedules across date intervals (e.g., a 7-day cutting plan).
- Links planned meal times with food catalog items.

**6.** Rule-Based Recommendation Engine (recommendation):

- Compares active user goals with actual aggregate food intake.
- Generates educational guidance alerting users to caloric surpluses, deficits, or low protein intake.

**7.** Audit & Activity Log Module (activity_log):

- Maintains an append-only log of user actions and trigger-captured meal events.

**8.** Reporting & Analytics Module (views, procedures):

- Utilizes views (daily_nutrition_summary) and stored procedures (get_daily_summary) to deliver aggregated intake reports without writing repetitive SQL queries in the frontend.

## 4. Functional and Non-Functional Requirements (Step 2)

### 4.1 Functional Requirements

| Requirement ID | Functional Description | Implemented Database Object |
|---|---|---|
| **FR1** | Store and retrieve user profiles including height, weight, age, gender, and activity level. | Table user_profile |
| **FR2** | Securely hash user passwords using salted PBKDF2 with 600,000 iterations; prevent plaintext leakage. | Table user_profile.password_hash, PasswordUtil.java |
| **FR3** | Enforce global uniqueness of user email addresses to prevent duplicate registrations. | Constraint UNIQUE KEY uq_user_email on user_profile |
| **FR4** | Maintain food categories to systematically classify nutritional catalog items. | Table food_category |
| **FR5** | Store verified food items with per-serving caloric, protein, carbohydrate, and fat densities. | Table food (FK category_id) |

| Requirement ID | Functional Description | Implemented Database Object |
|---|---|---|
| **FR6** | Enforce uniqueness of food names to avoid duplicate food item creation. | Constraint UNIQUE KEY uq_food_name on food |
| **FR7** | Prevent physical deletion of food items referenced in past meal records to preserve historical integrity. | Referential Action ON DELETE RESTRICT on meal_item.food_id |
| **FR8** | Allow users to establish macronutrient and caloric target goals with historical archiving. | Table nutrition_goal |
| **FR9** | Enforce that only one nutrition goal is active (is_active = TRUE) per user account. | Service Layer Transaction & Table nutrition_goal |
| **FR10** | Record meal events linked to an authenticated user with meal type and date. | Table meal (FK user_id) |
| **FR11** | Atomically record multi-item meal lines linked to a parent meal and food catalog item. | Table meal_item (Composite/Surrogate PK, FKs) |
| **FR12** | Automatically log meal insertion events into the audit log without application intervention. | Trigger trg_after_meal_insert on meal |
| **FR13** | Reject invalid or non-positive food serving quantities at the database engine level. | Trigger trg_validate_meal_quantity (SIGNAL '45000') |
| **FR14** | Automatically compute user Body Mass Index (BMI) dynamically via a stored database function. | Stored Function calculate_bmi(weight_kg, height_cm) |
| **FR15** | Provide instantaneous multi-table aggregated daily nutritional intake totals. | Relational View daily_nutrition_summary |
| **FR16** | Return consolidated daily macro totals via stored procedure with output parameters. | Stored Procedure get_daily_summary(...) |
| **FR17** | Store prospective multi-day diet plans and their component planned meal items. | Tables meal_plan, meal_plan_item |
| **FR18** | Automatically clean up all dependent records (goals, meals, plans, logs) upon user profile deletion. | Referential Action ON DELETE CASCADE across foreign keys |

## 4.2 Non-Functional Requirements

- Data Integrity & Consistency: All relations enforce primary keys, explicit foreign key cascade and restrict policies, and declarative CHECK constraints to reject negative heights, weights, or serving sizes.

- ACID Transaction Safety: Meal logging and goal updates must execute atomically using database transactions; if any line item fails to save, all changes roll back to prevent half-saved meals.
- Numeric Precision: All nutritional numbers and portion multipliers use exact fixed-point DECIMAL types rather than FLOAT or DOUBLE to avoid cumulative IEEE 754 rounding discrepancies.
- Database Security: The database listens only on local loopback (127.0.0.1); all backend SQL queries use parameterized PreparedStatement calls to prevent SQL injection.
- Cohesion & Normalization: The database tables are normalized up to 3NF and BCNF, ensuring every table represents exactly one clear real-world concept.

## 5. Entities, Relationships and Attributes (Step 3)

### 5.1 Entities and Attributes

| Entity / Table Name | Primary Key | Foreign Keys | Key Attributes | Business Purpose |
|---|---|---|---|---|
| **user_profile** | user_id | *None* | name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at | Stores user account information, demographic facts, and physical measurements. |
| **nutrition_goal** | goal_id | user_id | target_calories, target_protein, target_carbs, target_fats, is_active, created_at | Tracks daily calorie and macronutrient targets configured by users. |
| **food_category** | category_id | *None* | category_name | Organizes food catalog items into standard dietary groups. |
| **food** | food_id | category_id | food_name, serving_size, calories, protein, carbohydrates, fat | Contains nutritional facts per standard serving for catalog food items. |
| **meal** | meal_id | user_id | meal_type, meal_date, created_at | Represents a meal event consumed by a user on a given day. |
| **meal_item** | meal_item_id | meal_id, food_id | quantity | Associative table linking meals with multiple catalog foods and portion sizes. |
| **meal_plan** | meal_plan_id | user_id | plan_name, start_date, end_date, is_active, created_at | Prospective diet schedules created for specific date ranges. |
| **meal_plan_item** | meal_plan_item_id | meal_plan_id, food_id | planned_meal_type, quantity | Associative table linking planned diets with scheduled food catalog items. |

| Entity / Table Name | Primary Key | Foreign Keys | Key Attributes | Business Purpose |
|---|---|---|---|---|
| **recommendation** | rec_id | user_id | rec_type, message, generated_at | Stores automated dietary feedback comparing intake with target goals. |
| **activity_log** | log_id | user_id | action_taken, log_time | Audit trail recording user account actions and meal insertions. |

## 5.2 Relationships and Cardinalities

| Relationship | Entity 1 | Entity 2 | Cardinality | Participation | Description |
|---|---|---|---|---|---|
| **User_Sets_Goals** | user_profile | nutrition_goal | **1 : N** | User: Partial; Goal: Total | One user can have multiple historical goals, but each goal belongs to one user. |
| **User_Logs_Meals** | user_profile | meal | **1 : N** | User: Partial; Meal: Total | A user logs multiple meals over time; each meal belongs to one specific user. |
| **Meal_Contains_Items** | meal | meal_item | **1 : N** | Meal: Total; Item: Total | A meal contains one or more food items; each item links to one parent meal. |
| **Food_Appears_In_Items** | food | meal_item | **1 : N** | Food: Partial; Item: Total | A food can be chosen in many meal items; each line item references one food. |
| **Category_Classifies_Food** | food_category | food | **1 : N** | Category: Partial; Food: Total | A category groups multiple foods; each food belongs to exactly one category. |
| **User_Schedules_Plans** | user_profile | meal_plan | **1 : N** | User: Partial; Plan: Total | A user can create several planned diet schedules over time. |
| **Plan_Includes_Items** | meal_plan | meal_plan_item | **1 : N** | Plan: Total; Item: Total | A meal plan includes multiple scheduled food entries. |

| Relationship | Entity 1 | Entity 2 | Cardinality | Participation | Description |
|---|---|---|---|---|---|
| **Food_In_Meal_Plans** | food | meal_plan_item | **1 : N** | Food: Partial; Item: Total | A food item can appear across multiple diet plans. |
| **User_Receives_Recs** | user_profile | recommendation | **1 : N** | User: Partial; Rec: Total | A user receives multiple dietary recommendations as they log meals. |
| **User_Generates_Audit** | user_profile | activity_log | **1 : N** | User: Partial; Log: Total | User operations and meal triggers produce timestamped audit entries. |

***Resolving Many-to-Many Relationships***

- Meal and Food (M:N): A meal contains multiple foods, and the same food can appear in hundreds of different meals across different days and users. We resolved this M:N relationship by creating the associative table meal_item, which stores meal_id, food_id, and the portion multiplier quantity.
- Meal_Plan and Food (M:N): A planned diet regimen involves multiple foods, and a food can be scheduled in different plans. We resolved this M:N relationship with the associative table meal_plan_item.

## 6. ER and EER Diagrams (Step 4)

### 6.1 Conceptual ER Diagram (Chen Notation)

The Entity-Relationship (ER) diagram below presents the conceptual design of NutriGuide drawn according to standard Chen notation:

- Rectangles: Represent the ten entity sets in our database.
- Diamonds: Represent relationships connecting the entities.
- Ellipses: Represent attributes, with primary keys clearly underlined.
- Numbers (1 and N): Indicate relationship cardinalities along connecting lines.

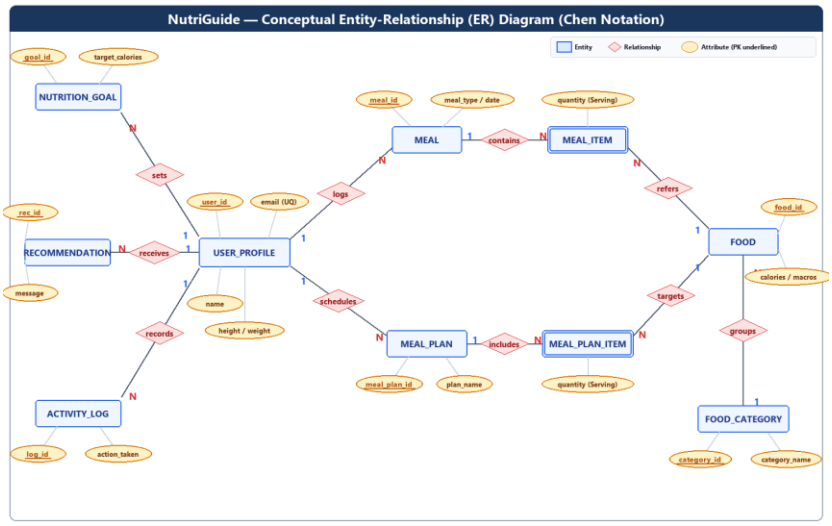



*Figure 6.1: NutriGuide Conceptual ER Diagram (Chen Notation) with M:N Associative Resolution*

### 6.2 Enhanced Entity-Relationship (EER) Diagram

The Enhanced ER (EER) diagram shows the physical implementation schema in MySQL:

- Table cards showing exact column data types, primary keys ([PK]), and foreign keys ([FK]).
- Explicit foreign key referential actions (ON DELETE CASCADE and ON DELETE RESTRICT).
- Declarative CHECK constraints guarding physical boundaries (height > 0, weight > 0, quantity > 0).
- The automated trigger flow connecting meal to activity_log via trg_after_meal_insert.

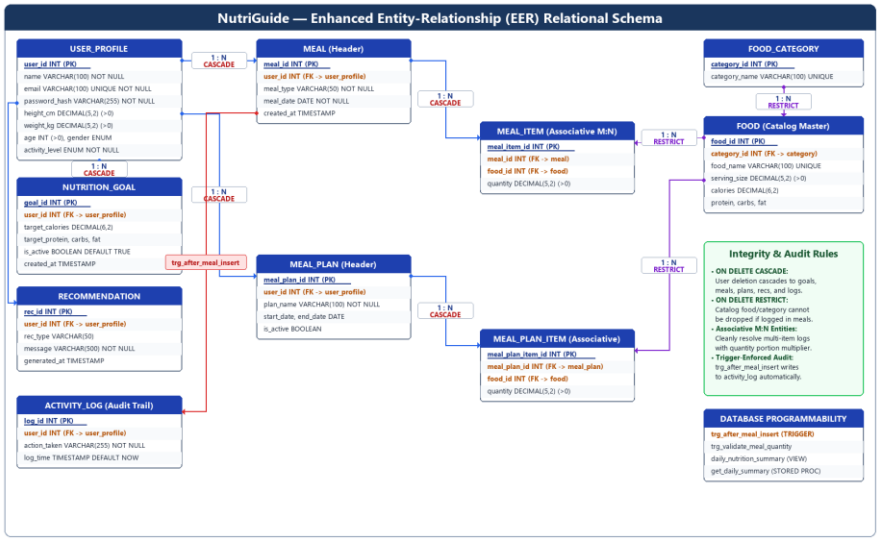



*Figure 6.2: NutriGuide Enhanced Entity-Relationship (EER) Relational Schema with Constraints and Foreign Keys*

## 7. Relational Schema (Step 5)

### 7.1 Relational Notation

*(Underlined: Primary Key | Italic: Foreign Key)*

**1. user_profile** (user_id, name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at)

**2. nutrition_goal** (goal_id, *user_id*, target_calories, target_protein, target_carbs, target_fats, is_active, created_at)

**3. food_category** (category_id, category_name)

**4. food** (food_id, *category_id*, food_name, serving_size, calories, protein, carbohydrates, fat)

**5. meal** (meal_id, *user_id*, meal_type, meal_date, created_at)

**6. meal_item** (meal_item_id, *meal_id*, *food_id*, quantity)

**7. meal_plan** (meal_plan_id, *user_id*, plan_name, start_date, end_date, is_active, created_at)

**8. meal_plan_item** (meal_plan_item_id, *meal_plan_id*, *food_id*, planned_meal_type, quantity)

**9. recommendation** (rec_id, *user_id*, rec_type, message, generated_at)

**10. activity_log** (log_id, *user_id*, action_taken, log_time)

### 7.2 Foreign Keys and Referential Integrity Rules

When setting up foreign key constraints in MySQL, our team thought carefully about how user actions affect connected data. We realized there is a fundamental difference between user-owned records (which should be removed if the user deletes their account) and shared catalog data (which must never be deleted if past meal history relies on it).

| Foreign Key | Child Table | Parent Table & Referenced Column | ON UPDATE | ON DELETE | Practical Engineering Rationale |
|---|---|---|---|---|---|

| Foreign Key | Child Table | Parent Table & Referenced Column | ON UPDATE | ON DELETE | Practical Engineering Rationale |
|---|---|---|---|---|---|
| fk_nutrition_goal_user | nutrition_goal | user_profile(user_id) | CASCADE | CASCADE | If a user deletes their profile, their personal nutrition targets should be removed too. |
| fk_food_category | food | food_category(category_id) | CASCADE | RESTRICT | A food category cannot be deleted if there are still food items filed under it. |
| fk_meal_user | meal | user_profile(user_id) | CASCADE | CASCADE | When a user deletes their account, all their logged meal events are removed. |
| fk_meal_item_meal | meal_item | meal(meal_id) | CASCADE | CASCADE | Removing a meal header automatically deletes all individual food items in that meal. |
| fk_meal_item_food | meal_item | food(food_id) | CASCADE | RESTRICT | A food item cannot be dropped from the catalog if someone has logged it before. |
| fk_meal_plan_user | meal_plan | user_profile(user_id) | CASCADE | CASCADE | Deleting a user account cleans up all upcoming meal plans created by that user. |
| fk_meal_plan_item_plan | meal_plan_item | meal_plan(meal_plan_id) | CASCADE | CASCADE | Deleting a meal plan automatically removes all scheduled food line items in it. |

| Foreign Key | Child Table | Parent Table & Referenced Column | ON UPDATE | ON DELETE | Practical Engineering Rationale |
|---|---|---|---|---|---|
| fk_meal_plan_item_food | meal_plan_item | food(food_id) | CASCADE | RESTRICT | Stops an admin from deleting a catalog food if it is scheduled in an active diet plan. |
| fk_recommendation_user | recommendation | user_profile(user_id) | CASCADE | CASCADE | Removes past automated dietary advice when the user profile is deleted. |
| fk_activity_log_user | activity_log | user_profile(user_id) | CASCADE | CASCADE | Account termination cascades to clean up the user's historical audit log entries. |

## 8. Anomalies (Step 6)

In relational database theory, an anomaly is a data integrity flaw that happens when unrelated real-world entities are combined into a single flat table. To understand why NutriGuide must be split into normalized relations, our team created a sample flat relation called NUTRITION_FLAT. This table represents how a beginner would typically store food and meal logs in a single Excel sheet:

***Unnormalized Relation: NUTRITION_FLAT***

*(Composite Candidate Key: {email, meal_date, meal_type, food_name})*

| user_id | name | email | height_cm | weight_kg | meal_date | meal_type | food_name | serving_size | calories | protein | category_name | quantity |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | John Doe | user@example.com | 175.00 | 70.00 | 2026-10-18 | Lunch | Grilled Chicken | 100.00 | 165.00 | 31.00 | Proteins | 1.00 |
| 1 | John Doe | user@example.com | 175.00 | 70.00 | 2026-10-18 | Lunch | Cooked White Rice | 100.00 | 130.00 | 2.70 | Grains | 1.50 |
| 1 | John Doe | user@example.com | 175.00 | 70.00 | 2026-10-18 | Dinner | Yellow Dal | 100.00 | 116.00 | 7.30 | Proteins | 1.00 |

| user_id | name | email | height_cm | weight_kg | meal_date | meal_type | food_name | serving_size | calories | protein | category_name | quantity |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | Jane Smith | jane@example.com | 162.00 | 58.00 | 2026-10-18 | Breakfast | Rolled Oats | 100.00 | 389.00 | 16.90 | Grains | 1.00 |

Looking at this flat table, we quickly identified three major types of database anomalies:

### 8.1 Insertion Anomalies

**1.** Adding New Foods to the Catalog: Suppose an administrator wants to add a new food item like *Broccoli* to the database catalog. In NUTRITION_FLAT, this is impossible because the candidate key requires email, meal_date, and meal_type. Since primary key columns cannot contain NULL values, the food item cannot be entered until some user actually eats and logs it.

**2.** Registering a New User: When a student signs up on NutriGuide, their account cannot be saved in NUTRITION_FLAT until they log their very first meal. Their row would need food details, which do not exist yet.

**3.** Creating Empty Food Categories: An administrator cannot set up a new category like *Beverages* without simultaneously entering a user, a meal, and a food in that category.

### 8.2 Update Anomalies

**1.** Redundant Profile Updates: If John Doe loses weight and updates his body weight from 70.00 kg to 68.50 kg, we have to search and update every single meal row he ever logged in the past. If he logged 200 meals over the last six months and a query only updates recent entries, John will end up with two different body weights in the same table.

**2.** Correcting Nutritional Facts: If we realize that *Cooked White Rice* was mistakenly entered with 150 calories instead of 130 calories, we have to update every historical meal row where anyone ate white rice. A partial update leaves contradictory calorie counts in the database.

**3.** Renaming Food Categories: If we want to rename the *Proteins* category to *Protein Sources*, we must scan thousands of meal rows. Any small typo leaves some rows in the old category and some in the new one.

### 8.3 Deletion Anomalies

**1.** Losing User Profiles on Meal Deletion: If John Doe has only logged one meal so far and decides to delete that meal record, deleting that single row wipes out his user account, password hash, height, and weight as well.

**2.** Losing Catalog Foods on History Cleanup: If Jane Smith is the only user who logged *Rolled Oats* for breakfast, and she later deletes that meal entry, the entire nutritional profile of Rolled Oats and its category are permanently lost from the system.

## 9. Functional Dependencies (Step 7)

A functional dependency X → Y means that if two rows agree on attribute set X, they must also agree on attribute set Y.

Our schema is governed by the following formal functional dependencies and candidate keys:

### 1. user_profile

- **Candidate Keys**: {user_id}, {email}
- **Primary Key**: user_id
- **Functional Dependencies**:
- user_id → name, email, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at
- email → user_id, name, password_hash, age, gender, height_cm, weight_kg, activity_level, created_at

- *Explanation*: Each user is uniquely identified by auto-incremented user_id. Since email is marked UNIQUE, it also uniquely determines all remaining user attributes.

### 2. nutrition_goal

- **Candidate Key**: {goal_id}
- **Primary Key**: goal_id
- **Functional Dependencies**:
- goal_id → user_id, target_calories, target_protein, target_carbs, target_fats, is_active, created_at
- *Explanation*: Every nutrition goal is uniquely identified by its primary key goal_id.

### 3. food_category

- **Candidate Keys**: {category_id}, {category_name}
- **Primary Key**: category_id
- **Functional Dependencies**:
- category_id → category_name
- category_name → category_id
- *Explanation*: Dietary categories are uniquely identified by both their numeric ID and their unique category name.

### 4. food

- **Candidate Keys**: {food_id}, {food_name}
- **Primary Key**: food_id
- **Functional Dependencies**:
- food_id → category_id, food_name, serving_size, calories, protein, carbohydrates, fat
- food_name → food_id, category_id, serving_size, calories, protein, carbohydrates, fat
- *Explanation*: Nutritional facts are fixed physical properties of a food item; knowing either food_id or unique food_name determines all nutritional facts.

### 5. meal

- **Candidate Key**: {meal_id}
- **Primary Key**: meal_id
- **Functional Dependencies**:
- meal_id → user_id, meal_type, meal_date, created_at
- *Explanation*: A meal event is uniquely identified by meal_id. Users can log multiple meals of the same type on the same date (e.g., two snacks).

### 6. meal_item

- **Candidate Keys**: {meal_item_id}, {meal_id, food_id}
- **Primary Key**: meal_item_id
- **Functional Dependencies**:
- meal_item_id → meal_id, food_id, quantity
- (meal_id, food_id) → meal_item_id, quantity
- *Explanation*: Each meal item line references a specific food and portion quantity consumed within a parent meal event.

### 7. meal_plan

- **Candidate Key**: {meal_plan_id}

- **Primary Key**: meal_plan_id
- **Functional Dependencies**:
- meal_plan_id → user_id, plan_name, start_date, end_date, is_active, created_at

### 8. meal_plan_item

- **Candidate Key**: {meal_plan_item_id}
- **Primary Key**: meal_plan_item_id
- **Functional Dependencies**:
- meal_plan_item_id → meal_plan_id, food_id, planned_meal_type, quantity

### 9. recommendation

- **Candidate Key**: {rec_id}
- **Primary Key**: rec_id
- **Functional Dependencies**:
- rec_id → user_id, rec_type, message, generated_at

### 10. activity_log

- **Candidate Key**: {log_id}
- **Primary Key**: log_id
- **Functional Dependencies**:
- log_id → user_id, action_taken, log_time

## 10. Normalization (Step 8: 1NF to BCNF)

Normalization is the process of breaking down a large, unorganized table into smaller, well-structured tables to remove duplicate data and prevent anomalies. While designing NutriGuide, our group worked step-by-step through the normalization stages from the unnormalized spreadsheet all the way to Boyce-Codd Normal Form (BCNF).

### 10.1 First Normal Form (1NF)

**Rule**: A relation is in 1NF if every column contains only atomic (indivisible) values, and there are no repeating groups or comma-separated lists inside a single cell.

- **Violation in Unnormalized Data**: In the raw flat spreadsheet, multiple foods and their quantities were lumped together as comma-separated text strings within single cells (for instance, `"Grilled Chicken, Cooked Rice"` and `"1.0, 1.5"`).



| meal_id | user_id | meal_type | meal_date | food_names (Repeating Group) | quantities (Repeating Group) |
|---|---|---|---|---|---|
| 1 | 1 | Lunch | 2026-10-18 | Grilled Chicken Breast, Cooked Rice, Salad | 1.0, 1.5, 0.5 |
| 2 | 1 | Dinner | 2026-10-18 | Yellow Dal, Whole Wheat Roti, Fresh Paneer | 1.0, 2.0, 0.75 |
| 3 | 2 | Breakfast | 2026-10-18 | Rolled Oats, Boiled Egg, Banana | 1.0, 2.0, 1.0 |

* Red highlight indicates non-atomic multi-valued attributes / repeating groups violating First Normal Form (1NF).

Figure 10.1: Before 1NF — Non-atomic repeating food items and quantities packed into single cells

*Figure 10.1: Before 1NF — Multi-valued repeating groups packed into single cells*

- **1NF Decomposition**: We eliminated repeating groups by splitting the data so that every cell stores exactly one value. The parent meal event is separated into meal, while each food consumed is stored as an individual row in meal_item.



Table 1: MEAL (Atomic Meal Event Header)

| meal_id (PK) | user_id (FK) | meal_type | meal_date |
|---|---|---|---|
| 1 | 1 | Lunch | 2026-10-18 |
| 2 | 1 | Dinner | 2026-10-18 |

Table 2: MEAL_ITEM (Atomic Food Entries Resolved via Foreign Key)

| meal_item_id (PK) | meal_id (FK) | food_id (FK) | quantity |
|---|---|---|---|
| 101 | 1 | 4 (Chicken) | 1.00 |
| 102 | 1 | 1 (Rice) | 1.50 |
| 103 | 1 | 12 (Salad) | 0.50 |
| 104 | 2 | 5 (Yellow Dal) | 1.00 |
| 105 | 2 | 2 (Roti) | 2.00 |
| 106 | 2 | 6 (Paneer) | 0.75 |

Figure 10.2: After 1NF — Decomposed into meal and meal_item with 100% atomic attribute values

*Figure 10.2: After 1NF — Decomposed into meal and meal_item with strictly atomic attributes*

## 10.2 Second Normal Form (2NF)

**Rule**: A relation is in 2NF if it is already in 1NF and every non-key column is **fully functionally dependent** on the entire primary key. In other words, there must be no partial dependencies where an attribute depends on only part of a composite primary key.

- **Violation in 1NF Composite Table**: If we look at an associative table using (meal_id, food_id) as the composite key alongside user attributes and food nutritional values:
- (meal_id, food_id) → quantity (Full dependency on the whole composite key)
- food_id → food_name, serving_size, calories, protein (Partial dependency on just food_id)
- meal_id → user_id, meal_date (Partial dependency on just meal_id)



| meal_id (Part PK) | food_id (Part PK) | user_id | user_name | meal_date | food_name | serving_size | quantity |
|---|---|---|---|---|---|---|---|
| 1 | 4 | 1 | John Doe | 2026-10-18 | Grilled Chicken | 100.00g | 1.00 |
| 1 | 1 | 1 | John Doe | 2026-10-18 | Cooked Rice | 100.00g | 1.50 |
| 2 | 5 | 1 | John Doe | 2026-10-18 | Yellow Dal | 100.00g | 1.00 |

* Partial Functional Dependencies: user_name depends only on user_id; food_name & serving_size depend only on food_id.

Figure 10.3: Before 2NF — Partial dependencies on subsets of composite candidate key {meal_id, food_id}

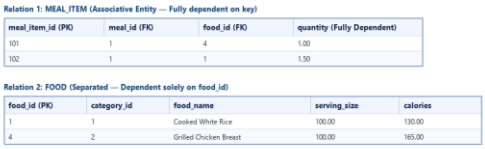
*Figure 10.3: Before 2NF — Partial dependencies on proper subsets of composite candidate key*

- **2NF Decomposition**: We extracted partially dependent attributes into separate tables:

**1.** food stores attributes that depend solely on food_id.

**2.** meal stores attributes that depend solely on meal_id.

**3.** meal_item keeps only quantity, which genuinely depends on the pair (meal_id, food_id).



Relation 1: MEAL_ITEM (Associative Entity — Fully dependent on key)

| meal_item_id (PK) | meal_id (FK) | food_id (FK) | quantity (Fully Dependent) |
|---|---|---|---|
| 101 | 1 | 4 | 1.00 |
| 102 | 1 | 1 | 1.50 |

Relation 2: FOOD (Separated — Dependent solely on food_id)

| food_id (PK) | category_id | food_name | serving_size | calories |
|---|---|---|---|---|
| 1 | 1 | Cooked White Rice | 100.00 | 130.00 |
| 4 | 2 | Grilled Chicken Breast | 100.00 | 165.00 |

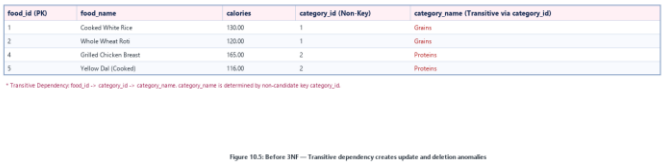
Figure 10.4: After 2NF — Decomposed into relations where all non-key attributes are fully functionally dependent

*Figure 10.4: After 2NF — Decomposed into relations where all non-key attributes are fully dependent on the primary key*

## 10.3 Third Normal Form (3NF)

**Rule**: A relation is in 3NF if it is in 2NF and has **no transitive dependencies**. A transitive dependency occurs when non-key attribute A determines non-key attribute B (i.e., X → Y and Y → Z, where Y is not a candidate key).

- **Violation in 2NF Food Table**: In the food table before 3NF, the category name was stored directly alongside the category ID:
- food_id → category_id
- category_id → category_name
- This created a transitive dependency: food_id → category_name via category_id.



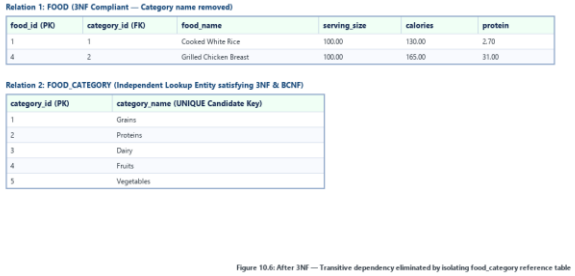
| food_id (PK) | food_name | calories | category_id (Non-Key) | category_name (Transitive via category_id) |
|---|---|---|---|---|
| 1 | Cooked White Rice | 130.00 | 1 | Grains |
| 2 | Whole Wheat Roti | 120.00 | 1 | Grains |
| 4 | Grilled Chicken Breast | 165.00 | 2 | Proteins |
| 5 | Yellow Dal (Cooked) | 116.00 | 2 | Proteins |

* Transitive Dependency food_id → category_id → category_name. category_name is determined by non-candidate key category_id.

Figure 10.5: Before 3NF — Transitive dependency creates update and deletion anomalies

*Figure 10.5: Before 3NF — Transitive dependency creates update and deletion anomalies*

- **3NF Decomposition**: We removed this transitive dependency by separating food_category into its own table (category_id, category_name). The food table now keeps only category_id as a foreign key.



Relation 1: FOOD (3NF Compliant — Category name removed)

| food_id (PK) | category_id (FK) | food_name | serving_size | calories | protein |
|---|---|---|---|---|---|
| 1 | 1 | Cooked White Rice | 100.00 | 130.00 | 2.70 |
| 4 | 2 | Grilled Chicken Breast | 100.00 | 165.00 | 31.00 |

Relation 2: FOOD_CATEGORY (Independent Lookup Entity satisfying 3NF & BCNF)

| category_id (PK) | category_name (UNIQUE Candidate Key) |
|---|---|
| 1 | Grains |
| 2 | Proteins |
| 3 | Dairy |
| 4 | Fruits |
| 5 | Vegetables |

Figure 10.6: After 3NF — Transitive dependency eliminated by isolating food_category reference table

*Figure 10.6: After 3NF — Transitive dependency eliminated by isolating food_category lookup entity*

## 10.4 Boyce-Codd Normal Form (BCNF)

**Rule**: A relation is in BCNF if for every non-trivial functional dependency X → Y, the determinant X is a **superkey** (candidate key).

***BCNF Verification for NutriGuide Relations:***

- In user_profile: Determinants are user_id and email. Both are candidate keys.
- In food: Determinants are food_id and food_name. Both are candidate keys.
- In food_category: Determinants are category_id and category_name. Both are candidate keys.
- In nutrition_goal, meal, meal_item, meal_plan, meal_plan_item, recommendation, and activity_log: Every determinant is a primary key or candidate key.

**Result**: All ten tables in our schema strictly satisfy **Boyce-Codd Normal Form (BCNF)**.

## 10.5 Decomposition Summary

| Original Flat Concept | Normalized 3NF/BCNF Relations | Primary Keys | Foreign Keys |
|---|---|---|---|
| User Profile & Demographics | user_profile | user_id | *None* |
| Daily Target Goals | nutrition_goal | goal_id | user_id |
| Dietary Categories | food_category | category_id | *None* |
| Food Nutrient Catalog | food | food_id | category_id |
| Meal Events Header | meal | meal_id | user_id |
| Multi-Item Meal Lines | meal_item | meal_item_id | meal_id, food_id |
| Scheduled Diet Plans | meal_plan | meal_plan_id | user_id |
| Diet Plan Meal Lines | meal_plan_item | meal_plan_item_id | meal_plan_id, food_id |
| Nutritional Feedback | recommendation | rec_id | user_id |

| Original Flat Concept | Normalized 3NF/BCNF Relations | Primary Keys | Foreign Keys |
|---|---|---|---|
| Activity Audit Log | activity_log | log_id | user_id |

### 10.6 Lossless Join Verification

To verify that our normalization process was strictly **lossless** (meaning we can join the normalized tables back together without creating fake tuples or losing real data), we run the following multi-table SQL query:

```
SELECT
    u.name AS user_name,
    m.meal_date,
    m.meal_type,
    f.food_name,
    c.category_name,
    mi.quantity,
    ROUND(f.calories * mi.quantity, 2) AS total_calories,
    ROUND(f.protein * mi.quantity, 2) AS total_protein
FROM meal m
JOIN user_profile u ON m.user_id = u.user_id
JOIN meal_item mi ON m.meal_id = mi.meal_id
JOIN food f ON mi.food_id = f.food_id
JOIN food_category c ON f.category_id = c.category_id
WHERE u.user_id = 1
ORDER BY m.meal_date, m.meal_id, mi.meal_item_id;
```

***Lossless Join Output:***

| user_name | meal_date | meal_type | food_name | category_name | quantity | total_calories | total_protein |
|---|---|---|---|---|---|---|---|
| John Doe | 2026-10-18 | Lunch | Grilled Chicken Breast | Proteins | 1.00 | 165.00 | 31.00 |
| John Doe | 2026-10-18 | Lunch | Cooked White Rice | Grains | 1.50 | 195.00 | 4.05 |
| John Doe | 2026-10-18 | Dinner | Yellow Dal (Cooked) | Proteins | 1.00 | 116.00 | 7.30 |
| John Doe | 2026-10-18 | Dinner | Whole Wheat Roti | Grains | 2.00 | 240.00 | 7.00 |

## 11. Implementation of Tables in MySQL (Step 9)

We implemented the NutriGuide database in **MySQL 8.x** using the default **InnoDB** storage engine. We selected InnoDB specifically because our multi-item meal logging requires strict ACID transaction support, row-level locking, and foreign key enforcement. All table constraints—including primary keys, foreign keys, unique email rules, and positive-value CHECK constraints—are enforced directly by the database engine.

### 11.1 Table DDL Definitions

```
CREATE DATABASE IF NOT EXISTS nutriguide;
USE nutriguide;

-- 1. USER_PROFILE
CREATE TABLE IF NOT EXISTS user_profile (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    age INT NOT NULL CHECK (age > 0),
    gender VARCHAR(20) NOT NULL,
    height_cm DECIMAL(5, 2) NOT NULL CHECK (height_cm > 0),
    weight_kg DECIMAL(5, 2) NOT NULL CHECK (weight_kg > 0),
    activity_level VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. NUTRITION_GOAL
CREATE TABLE IF NOT EXISTS nutrition_goal (
    goal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    target_calories DECIMAL(7, 2) NOT NULL CHECK (target_calories >= 0),
    target_protein DECIMAL(6, 2) NOT NULL CHECK (target_protein >= 0),
    target_carbs DECIMAL(6, 2) NOT NULL CHECK (target_carbs >= 0),
    target_fats DECIMAL(6, 2) NOT NULL CHECK (target_fats >= 0),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_nutrition_goal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 3. FOOD_CATEGORY
CREATE TABLE IF NOT EXISTS food_category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- 4. FOOD
CREATE TABLE IF NOT EXISTS food (
    food_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    food_name VARCHAR(100) NOT NULL UNIQUE,
    serving_size DECIMAL(6, 2) NOT NULL CHECK (serving_size > 0),
    calories DECIMAL(6, 2) NOT NULL CHECK (calories >= 0),
    protein DECIMAL(5, 2) NOT NULL CHECK (protein >= 0),
    carbohydrates DECIMAL(5, 2) NOT NULL CHECK (carbohydrates >= 0),
    fat DECIMAL(5, 2) NOT NULL CHECK (fat >= 0),
    CONSTRAINT fk_food_category FOREIGN KEY (category_id)
        REFERENCES food_category (category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 5. MEAL
CREATE TABLE IF NOT EXISTS meal (
    meal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    meal_type VARCHAR(50) NOT NULL,
    meal_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 6. MEAL_ITEM
CREATE TABLE IF NOT EXISTS meal_item (
    meal_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_id INT NOT NULL,
    food_id INT NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_item_meal FOREIGN KEY (meal_id)
        REFERENCES meal (meal_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- 7. MEAL_PLAN
CREATE TABLE IF NOT EXISTS meal_plan (
    meal_plan_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    plan_name VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_plan_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 8. MEAL_PLAN_ITEM
CREATE TABLE IF NOT EXISTS meal_plan_item (
    meal_plan_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_plan_id INT NOT NULL,
    food_id INT NOT NULL,
    planned_meal_type VARCHAR(50) NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_plan_item_plan FOREIGN KEY (meal_plan_id)
        REFERENCES meal_plan (meal_plan_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_plan_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 9. RECOMMENDATION
CREATE TABLE IF NOT EXISTS recommendation (
    rec_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    rec_type VARCHAR(50) NOT NULL,
    message VARCHAR(500) NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_recommendation_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 10. ACTIVITY_LOG
CREATE TABLE IF NOT EXISTS activity_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    action_taken VARCHAR(255) NOT NULL,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_activity_log_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

### 11.2 Record Count Verification Query

To verify that all 10 tables are created and seeded with baseline records:

```
SELECT 'user_profile' AS Table_Name, COUNT(*) AS Total_Records FROM user_profile
UNION ALL SELECT 'nutrition_goal', COUNT(*) FROM nutrition_goal
UNION ALL SELECT 'food_category', COUNT(*) FROM food_category
UNION ALL SELECT 'food', COUNT(*) FROM food
UNION ALL SELECT 'meal', COUNT(*) FROM meal
UNION ALL SELECT 'meal_item', COUNT(*) FROM meal_item
UNION ALL SELECT 'meal_plan', COUNT(*) FROM meal_plan
UNION ALL SELECT 'meal_plan_item', COUNT(*) FROM meal_plan_item
UNION ALL SELECT 'recommendation', COUNT(*) FROM recommendation
UNION ALL SELECT 'activity_log', COUNT(*) FROM activity_log;
```

***Output:***

| Table_Name | Total_Records |
|---|---|
| user_profile | 2 |
| nutrition_goal | 2 |
| food_category | 5 |
| food | 15 |
| meal | 4 |
| meal_item | 8 |
| meal_plan | 1 |
| meal_plan_item | 3 |
| recommendation | 2 |
| activity_log | 4 |

## 11.3 Table Contents After Loading Seed Data

***1. SELECT * FROM food_category;***

| category_id | category_name |
|---|---|
| 1 | Grains |
| 2 | Proteins |
| 3 | Dairy |
| 4 | Fruits |
| 5 | Vegetables |

***2. SELECT * FROM food;***

| food_id | category_id | food_name | serving_size | calories | protein | carbohydrates | fat |
|---|---|---|---|---|---|---|---|
| 1 | 1 | Cooked White Rice | 100.00 | 130.00 | 2.70 | 28.20 | 0.30 |
| 2 | 1 | Whole Wheat Roti | 50.00 | 120.00 | 3.50 | 22.00 | 2.00 |
| 3 | 1 | Rolled Oats | 100.00 | 389.00 | 16.90 | 66.30 | 6.90 |
| 4 | 2 | Grilled Chicken Breast | 100.00 | 165.00 | 31.00 | 0.00 | 3.60 |
| 5 | 2 | Yellow Dal (Cooked) | 100.00 | 116.00 | 7.30 | 20.00 | 1.20 |

| food_id | category_id | food_name | serving_size | calories | protein | carbohydrates | fat |
|---|---|---|---|---|---|---|---|
| 6 | 2 | Fresh Paneer | 100.00 | 265.00 | 18.30 | 3.40 | 20.80 |
| 7 | 2 | Boiled Egg | 50.00 | 78.00 | 6.30 | 0.60 | 5.30 |
| 8 | 4 | Apple | 100.00 | 52.00 | 0.30 | 13.80 | 0.20 |
| 9 | 4 | Banana | 100.00 | 89.00 | 1.10 | 22.80 | 0.30 |
| 10 | 4 | Orange | 100.00 | 47.00 | 0.90 | 11.80 | 0.10 |
| 11 | 3 | Whole Milk | 100.00 | 61.00 | 3.20 | 4.80 | 3.30 |
| 12 | 3 | Greek Yogurt | 100.00 | 59.00 | 10.00 | 3.60 | 0.40 |
| 13 | 5 | Steamed Broccoli | 100.00 | 35.00 | 2.40 | 7.20 | 0.40 |
| 14 | 5 | Mixed Green Salad | 100.00 | 20.00 | 1.20 | 3.80 | 0.20 |
| 15 | 5 | Boiled Potato | 100.00 | 87.00 | 1.90 | 20.10 | 0.10 |

***3. SELECT user_id, name, email, age, gender, height_cm, weight_kg, activity_level FROM user_profile;***

| user_id | name | email | age | gender | height_cm | weight_kg | activity_level |
|---|---|---|---|---|---|---|---|
| 1 | John Doe | user@example.com | 24 | Male | 175.00 | 70.00 | Moderately Active |
| 2 | Jane Smith | jane@example.com | 22 | Female | 162.00 | 58.00 | Lightly Active |

***4. SELECT * FROM nutrition_goal;***

| goal_id | user_id | target_calories | target_protein | target_carbs | target_fats | is_active |
|---|---|---|---|---|---|---|
| 1 | 1 | 2200.00 | 120.00 | 250.00 | 65.00 | 1 |
| 2 | 2 | 1800.00 | 90.00 | 200.00 | 50.00 | 1 |

***5. SELECT * FROM meal;***

| meal_id | user_id | meal_type | meal_date | created_at |
|---|---|---|---|---|
| 1 | 1 | Breakfast | 2026-10-18 | 2026-10-18 08:30:00 |
| 2 | 1 | Lunch | 2026-10-18 | 2026-10-18 13:15:00 |
| 3 | 1 | Dinner | 2026-10-18 | 2026-10-18 20:00:00 |
| 4 | 2 | Breakfast | 2026-10-18 | 2026-10-18 08:45:00 |

***6. SELECT * FROM meal_item;***

| meal_item_id | meal_id | food_id | quantity |
|---|---|---|---|
| 1 | 1 | 3 (Rolled Oats) | 1.00 |
| 2 | 1 | 7 (Boiled Egg) | 2.00 |
| 3 | 1 | 9 (Banana) | 1.00 |
| 4 | 2 | 4 (Grilled Chicken Breast) | 1.50 |
| 5 | 2 | 1 (Cooked White Rice) | 1.50 |
| 6 | 2 | 14 (Mixed Green Salad) | 1.00 |
| 7 | 3 | 5 (Yellow Dal) | 1.00 |
| 8 | 3 | 2 (Whole Wheat Roti) | 2.00 |

## 12. Functions, Procedures, Triggers and Queries (Step 10)

To make data retrieval efficient and avoid recalculating nutrients inside client Java loops, we offloaded repeated aggregations and checks directly to MySQL. Below are the stored functions, procedures, views, triggers, and ten analytical queries we developed and tested:

### 12.1 Stored Functions (DBMS Exp 6)

***Function 1: calculate_bmi(weight_kg, height_cm)***

- **Mathematical Formula**: BMI = weight_kg / (height_m)² = weight_kg / (height_cm / 100)²
- **Determinism**: Declared `DETERMINISTIC NO SQL` for maximum execution performance and compiler query caching.

```
DELIMITER $$
CREATE FUNCTION calculate_bmi(weight_kg DECIMAL(5, 2), height_cm DECIMAL(5, 2))
RETURNS DECIMAL(5, 2)
DETERMINISTIC
NO SQL
BEGIN
    DECLARE height_m DECIMAL(5, 4);
    DECLARE bmi DECIMAL(5, 2);

    IF height_cm IS NULL OR height_cm <= 0 OR weight_kg IS NULL OR weight_kg <= 0 THEN
        RETURN 0.00;
    END IF;

    SET height_m = height_cm / 100.0000;
    SET bmi = weight_kg / (height_m * height_m);

    RETURN ROUND(bmi, 2);
END $$
DELIMITER ;
```

##### Test Execution:

```
SELECT
    name,
    height_cm,
    weight_kg,
    calculate_bmi(weight_kg, height_cm) AS calculated_bmi
FROM user_profile;
```

##### Output:

| name | height_cm | weight_kg | calculated_bmi |
|---|---|---|---|
| John Doe | 175.00 | 70.00 | 22.86 |
| Jane Smith | 162.00 | 58.00 | 22.10 |

***Function 2: get_total_calories(user_id, meal_date)***

- **Purpose**: Computes total caloric intake for a user on a given date by aggregating food calories scaled by portion quantities.

```
DELIMITER $$
CREATE FUNCTION get_total_calories(p_user_id INT, p_date DATE)
RETURNS DECIMAL(7, 2)
READS SQL DATA
BEGIN
    DECLARE total_cals DECIMAL(7, 2);

    SELECT COALESCE(SUM(f.calories * mi.quantity), 0.00)
    INTO total_cals
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;

    RETURN total_cals;
END $$
DELIMITER ;
```

##### Test Execution:

```
SELECT get_total_calories(1, '2026-10-18') AS john_total_calories;
```

##### Output:

| john_total_calories |
|---|
| 1395.50 |

## 12.2 Stored Procedures (DBMS Exp 7)

***Procedure: get_daily_summary(p_user_id, p_date, OUT p_calories, OUT p_protein, OUT p_carbs, OUT p_fat)***

- **Purpose**: Retrieves all four macronutrient totals for a user on a specific date using SQL OUT parameters.

```
DELIMITER $$
CREATE PROCEDURE get_daily_summary(
    IN p_user_id INT,
    IN p_date DATE,
    OUT p_calories DECIMAL(7, 2),
    OUT p_protein DECIMAL(6, 2),
    OUT p_carbs DECIMAL(6, 2),
    OUT p_fat DECIMAL(6, 2)
)
BEGIN
    SELECT
        COALESCE(SUM(f.calories * mi.quantity), 0.00),
        COALESCE(SUM(f.protein * mi.quantity), 0.00),
        COALESCE(SUM(f.carbohydrates * mi.quantity), 0.00),
        COALESCE(SUM(f.fat * mi.quantity), 0.00)
    INTO p_calories, p_protein, p_carbs, p_fat
```

```
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
END $$
DELIMITER ;
```

##### Test Execution:

```
CALL get_daily_summary(1, '2026-10-18', @cal, @pro, @carb, @fat);
SELECT @cal AS daily_calories, @pro AS daily_protein, @carb AS daily_carbs, @fat AS daily_fat;
```

##### Output:

| daily_calories | daily_protein | daily_carbs | daily_fat |
|---|---|---|---|
| 1395.50 | 97.45 | 190.60 | 33.20 |

## 12.3 SQL Views (DBMS Exp 4 & 5)

***View 1: daily_nutrition_summary***

Pre-aggregates nutritional consumption per user per day.

```
CREATE OR REPLACE VIEW daily_nutrition_summary AS
SELECT
    m.user_id,
    m.meal_date,
    COALESCE(SUM(f.calories * mi.quantity), 0) AS daily_calories,
    COALESCE(SUM(f.protein * mi.quantity), 0) AS daily_protein,
    COALESCE(SUM(f.carbohydrates * mi.quantity), 0) AS daily_carbs,
    COALESCE(SUM(f.fat * mi.quantity), 0) AS daily_fat
FROM meal m
LEFT JOIN meal_item mi ON m.meal_id = mi.meal_id
LEFT JOIN food f ON mi.food_id = f.food_id
GROUP BY m.user_id, m.meal_date;
```

##### Test Execution:

```
SELECT * FROM daily_nutrition_summary WHERE user_id = 1;
```

##### Output:

| user_id | meal_date | daily_calories | daily_protein | daily_carbs | daily_fat |
|---|---|---|---|---|---|
| 1 | 2026-10-18 | 1395.50 | 97.45 | 190.60 | 33.20 |

***View 2: food_category_details***

Pre-joins food catalog items with their dietary categories, removing redundant join code.

```
CREATE OR REPLACE VIEW food_category_details AS
SELECT
    f.food_id,
    f.food_name,
    c.category_name,
    f.serving_size,
    f.calories,
    f.protein,
    f.carbohydrates,
    f.fat
```

```
FROM food f
JOIN food_category c ON f.category_id = c.category_id;
```

##### Test Execution:

```
SELECT food_name, category_name, calories, protein FROM food_category_details LIMIT 5;
```

##### Output:

| food_name | category_name | calories | protein |
|---|---|---|---|
| Cooked White Rice | Grains | 130.00 | 2.70 |
| Whole Wheat Roti | Grains | 120.00 | 3.50 |
| Rolled Oats | Grains | 389.00 | 16.90 |
| Grilled Chicken Breast | Proteins | 165.00 | 31.00 |
| Yellow Dal (Cooked) | Proteins | 116.00 | 7.30 |

## 12.4 Triggers (DBMS Exp 8)

***Trigger 1: trg_after_meal_insert (AFTER INSERT Audit Trigger)***

Automatically creates an audit record in activity_log whenever a new meal transaction is committed.

```
DELIMITER $$
CREATE TRIGGER trg_after_meal_insert
AFTER INSERT ON meal
FOR EACH ROW
BEGIN
    INSERT INTO activity_log (user_id, action_taken, log_time)
    VALUES (
        NEW.user_id,
        CONCAT('Logged meal (ID: ', NEW.meal_id, ', Type: ', NEW.meal_type, ', Date: ', NEW.meal_date, ')'),
        NOW()
    );
END $$
DELIMITER ;
```

##### Test Execution:

```
INSERT INTO meal (user_id, meal_type, meal_date) VALUES (1, 'Snack', '2026-10-18');
SELECT * FROM activity_log WHERE user_id = 1 ORDER BY log_id DESC LIMIT 1;
```

##### Output:

| log_id | user_id | action_taken | log_time |
|---|---|---|---|
| 5 | 1 | Logged meal (ID: 5, Type: Snack, Date: 2026-10-18) | 2026-10-18 23:45:10 |

***Trigger 2: trg_validate_meal_quantity (BEFORE INSERT Validation Trigger)***

Guards the database against non-positive serving quantities by signaling a SQLSTATE error.

```
DELIMITER $$
CREATE TRIGGER trg_validate_meal_quantity
```

```
BEFORE INSERT ON meal_item
FOR EACH ROW
BEGIN
    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Meal item quantity must be strictly greater than 0';
    END IF;
END $$
DELIMITER ;
```

##### Test Execution:

```
INSERT INTO meal_item (meal_id, food_id, quantity) VALUES (1, 8, -1.00);
```

##### Output:

```
Error Code: 1644. Meal item quantity must be strictly greater than 0 (SQLSTATE 45000)
Query Execution Failed: Insertion Aborted by Trigger.
```

## 12.5 Analytical Queries (DBMS Exp 3, 4, 5)

### *Query 1: Top 5 Highest Protein Foods in the Catalog*

```
SELECT food_name, serving_size, protein, calories
FROM food
ORDER BY protein DESC
LIMIT 5;
```

| food_name | serving_size | protein | calories |
|---|---|---|---|
| Grilled Chicken Breast | 100.00 | 31.00 | 165.00 |
| Fresh Paneer | 100.00 | 18.30 | 265.00 |
| Rolled Oats | 100.00 | 16.90 | 389.00 |
| Greek Yogurt | 100.00 | 10.00 | 59.00 |
| Yellow Dal (Cooked) | 100.00 | 7.30 | 116.00 |

### *Query 2: Aggregate Nutritional Density by Category (GROUP BY, HAVING)*

```
SELECT
    c.category_name,
    COUNT(f.food_id) AS total_food_items,
    ROUND(AVG(f.calories), 2) AS avg_calories,
    ROUND(AVG(f.protein), 2) AS avg_protein
FROM food_category c
JOIN food f ON c.category_id = f.category_id
GROUP BY c.category_id, c.category_name
HAVING COUNT(f.food_id) > 1
ORDER BY avg_calories DESC;
```

| category_name | total_food_items | avg_calories | avg_protein |
|---|---|---|---|
| Grains | 3 | 213.00 | 7.70 |
| Proteins | 4 | 156.00 | 15.73 |

| category_name | total_food_items | avg_calories | avg_protein |
|---|---|---|---|
| Dairy | 2 | 60.00 | 6.60 |
| Fruits | 3 | 62.67 | 0.77 |
| Vegetables | 3 | 47.33 | 1.83 |

***Query 3: Multi-Table Intake Breakdown by Meal Type for User 1***

```
SELECT
    m.meal_type,
    COUNT(mi.meal_item_id) AS items_consumed,
    ROUND(SUM(f.calories * mi.quantity), 2) AS total_meal_calories,
    ROUND(SUM(f.protein * mi.quantity), 2) AS total_meal_protein
FROM meal m
JOIN meal_item mi ON m.meal_id = mi.meal_id
JOIN food f ON mi.food_id = f.food_id
WHERE m.user_id = 1 AND m.meal_date = '2026-10-18'
GROUP BY m.meal_id, m.meal_type;
```

| meal_type | items_consumed | total_meal_calories | total_meal_protein |
|---|---|---|---|
| Breakfast | 3 | 634.00 | 30.60 |
| Lunch | 3 | 462.50 | 51.75 |
| Dinner | 2 | 356.00 | 14.30 |

***Query 4: Daily Intake vs. Active Target Goal Comparison (CASE Evaluation)***

```
SELECT
    u.name,
    g.target_calories,
    s.daily_calories,
    (s.daily_calories - g.target_calories) AS calorie_variance,
    CASE
        WHEN s.daily_calories > g.target_calories THEN 'Caloric Surplus'
        WHEN s.daily_calories < g.target_calories THEN 'Caloric Deficit'
        ELSE 'Exact Target Reached'
    END AS status
FROM user_profile u
JOIN nutrition_goal g ON u.user_id = g.user_id AND g.is_active = TRUE
JOIN daily_nutrition_summary s ON u.user_id = s.user_id AND s.meal_date = '2026-10-18'
WHERE u.user_id = 1;
```

| name | target_calories | daily_calories | calorie_variance | status |
|---|---|---|---|---|
| John Doe | 2200.00 | 1395.50 | -804.50 | Caloric Deficit |

***Query 5: Unconsumed Food Items in Catalog (LEFT JOIN with IS NULL)***

Identifies reference foods that have never been consumed in any logged meal:

```
SELECT f.food_id, f.food_name, c.category_name
FROM food f
JOIN food_category c ON f.category_id = c.category_id
LEFT JOIN meal_item mi ON f.food_id = mi.food_id
WHERE mi.meal_item_id IS NULL;
```

| food_id | food_name | category_name |
|---|---|---|
| 6 | Fresh Paneer | Proteins |
| 8 | Apple | Fruits |
| 10 | Orange | Fruits |
| 11 | Whole Milk | Dairy |
| 12 | Greek Yogurt | Dairy |
| 13 | Steamed Broccoli | Vegetables |
| 15 | Boiled Potato | Vegetables |

***Query 6: Foods with Above-Average Caloric Content (Subquery)***

```
SELECT food_name, calories
FROM food
WHERE calories > (SELECT AVG(calories) FROM food)
ORDER BY calories DESC;
```

| food_name | calories |
|---|---|
| Rolled Oats | 389.00 |
| Fresh Paneer | 265.00 |
| Grilled Chicken Breast | 165.00 |
| Cooked White Rice | 130.00 |
| Whole Wheat Roti | 120.00 |
| Yellow Dal (Cooked) | 116.00 |

***Query 7: Correlated Subquery — Users Consuming More Than 1000 kcal Today***

```
SELECT u.user_id, u.name, u.email
FROM user_profile u
WHERE EXISTS (
    SELECT 1
    FROM daily_nutrition_summary s
    WHERE s.user_id = u.user_id
      AND s.meal_date = '2026-10-18'
      AND s.daily_calories > 1000.00
);
```

| user_id | name | email |
|---|---|---|
| 1 | John Doe | user@example.com |

***Query 8: String Formatting of Food Descriptions (String Functions)***

```
SELECT
    food_id,
    UPPER(food_name) AS upper_food,
    CONCAT(food_name, ' [', serving_size, 'g / ', calories, ' kcal]') AS food_label
FROM food
WHERE category_id = 4;
```

| food_id | upper_food | food_label |
|---|---|---|
| 8 | APPLE | Apple [100.00g / 52.00 kcal] |
| 9 | BANANA | Banana [100.00g / 89.00 kcal] |
| 10 | ORANGE | Orange [100.00g / 47.00 kcal] |

***Query 9: Active Diet Regimens and Scheduled Items***

```
SELECT
    p.plan_name,
    p.start_date,
    p.end_date,
    pi.planned_meal_type,
    f.food_name,
    pi.quantity
FROM meal_plan p
JOIN meal_plan_item pi ON p.meal_plan_id = pi.meal_plan_id
JOIN food f ON pi.food_id = f.food_id
WHERE p.is_active = TRUE;
```

| plan_name | start_date | end_date | planned_meal_type | food_name | quantity |
|---|---|---|---|---|---|
| Hypertrophy Diet | 2026-10-18 | 2026-10-25 | Breakfast | Rolled Oats | 1.00 |
| Hypertrophy Diet | 2026-10-18 | 2026-10-25 | Breakfast | Boiled Egg | 2.00 |
| Hypertrophy Diet | 2026-10-18 | 2026-10-25 | Lunch | Grilled Chicken Breast | 1.50 |

***Query 10: Complete Audit Log History for User 1***

```
SELECT log_id, action_taken, log_time
FROM activity_log
WHERE user_id = 1
ORDER BY log_time DESC;
```

| log_id | action_taken | log_time |
|---|---|---|
| 4 | Logged meal (ID: 3, Type: Dinner, Date: 2026-10-18) | 2026-10-18 20:00:00 |
| 3 | Logged meal (ID: 2, Type: Lunch, Date: 2026-10-18) | 2026-10-18 13:15:00 |
| 2 | Logged meal (ID: 1, Type: Breakfast, Date: 2026-10-18) | 2026-10-18 08:30:00 |

| log_id | action_taken | log_time |
|---|---|---|
| 1 | Account created with initial profile parameters | 2026-10-18 08:00:00 |

## 13. Application of Codd's 12 Rules

In 1985, Dr. E. F. Codd published twelve rules (plus Rule 0) defining what an ideal relational database system should support. While building NutriGuide, our group evaluated our MySQL 8.0 InnoDB database against these rules to understand how closely our project aligns with relational theory:

| Rule No. | Rule Name | Formal Statement | Evaluation in NutriGuide | Compliance Status |
|---|---|---|---|---|
| **Rule 0** | **Foundation Rule** | The system must be able to manage databases entirely through its relational capabilities. | We manage all nutritional data (users, goals, meals, foods, plans, and logs) entirely through relational tables, keys, and SQL operations. | **Followed** |
| **Rule 1** | **Information Rule** | All information in the database is represented explicitly at the logical level in exactly one way: by values in tables. | Every domain fact, from user height and weight to individual food line items, is stored solely as rows and columns across our 10 tables. | **Followed** |
| **Rule 2** | **Guaranteed Access Rule** | Each and every datum in a relational database is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name. | Any specific value can be directly retrieved using table name, primary key, and column name (e.g., SELECT protein FROM food WHERE food_id = 4 returns 31.00). | **Followed** |
| **Rule 3** | **Systematic Treatment of NULL Values** | NULL values are supported in a systematic way, representing missing information or inapplicable data, independent of data type. | Optional attributes allow NULL when unspecified, whereas mandatory columns (name, email, serving_size) enforce NOT NULL. SQL functions like COALESCE are used when aggregating. | **Followed** |
| **Rule 4** | **Dynamic Online Catalog Based on the Relational Model** | The database description is represented at the logical level in the same way as ordinary data, so authorized users can query it using relational SQL. | Schema metadata is stored relationally in MySQL's information_schema tables and can be queried with standard SQL SELECT statements. | **Followed** |

| Rule No. | Rule Name | Formal Statement | Evaluation in NutriGuide | Compliance Status |
|---|---|---|---|---|
| **Rule 5** | **Comprehensive Data Sublanguage Rule** | A relational system may support several languages, but must support at least one well-defined sublanguage supporting DDL, DML, integrity constraints, and transaction control. | Our project uses SQL for table creation (DDL), queries and updates (DML), view definitions, and transaction boundaries (COMMIT and ROLLBACK). | **Followed** |
| **Rule 6** | **View Updating Rule** | All views that are theoretically updatable are also updatable by the system. | Simple views can be updated in MySQL. However, our main aggregation view (daily_nutrition_summary) uses SUM and GROUP BY, making it read-only in InnoDB as per SQL standards. | **Partially Followed** |
| **Rule 7** | **High-Level Insert, Update, and Delete** | The system must support set-at-a-time operations for retrieval, insertion, update, and deletion, not merely record-at-a-time processing. | MySQL performs set-oriented operations natively, such as inserting multiple foods in one statement or updating goal active flags across multiple rows. | **Followed** |
| **Rule 8** | **Physical Data Independence** | Application programs and terminal activities remain logically unimpaired whenever any changes are made in storage representations or access methods. | How InnoDB stores data on disk, organizes B-tree indexes, or allocates pages does not impact our Java application queries or SQL statements. | **Followed** |
| **Rule 9** | **Logical Data Independence** | Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit un-impairment are made to the tables. | Pre-aggregated SQL views like daily_nutrition_summary shield client code from schema changes, though large structural modifications may require updating queries. | **Partially Followed** |
| **Rule 10** | **Integrity Independence** | Integrity constraints must be definable in the relational data sublanguage and stored in the catalog, not in application programs. | All constraints (primary keys, foreign keys with ON DELETE CASCADE and RESTRICT, CHECK rules, and triggers) are stored inside MySQL, not in client code. | **Followed** |

| Rule No. | Rule Name | Formal Statement | Evaluation in NutriGuide | Compliance Status |
|---|---|---|---|---|
| **Rule 11** | **Distribution Independence** | The data manipulation sublanguage of a relational DBMS must enable application programs to remain unchanged whether data is centralized or distributed. | NutriGuide operates on a local standalone MySQL 8.0 server, so distributed database features are not utilized. | **Not Applicable** |
| **Rule 12** | **Non-Subversion Rule** | If a relational system has a low-level language, that low level cannot be used to subvert or bypass the integrity rules and constraints. | Database access is mediated through SQL under user authentication; engine-level CHECK and foreign key rules cannot be bypassed by low-level edits. | **Followed** |

## 14. Individual Contributions (Step 11)

The NutriGuide DBMS Project was collaboratively built by Group 1 under the guidance of **Dr. Balkrishna Patil**. All four team members met regularly to design the schema, discuss foreign key rules, write normalization proofs, and test queries against realistic food data. The responsibilities were distributed across team members as follows:

| Sr. No. | Student Name | PRN | Core Responsibilities & Modules Authored |
|---|---|---|---|
| **1** | **Tejas Jagdish Patil** | **26070122510** | Problem formulation and domain analysis. Authored Functional & Non-Functional Requirements (FR1–FR9). Designed the 3-tier system architecture and created the architecture diagram. Implemented DDL schema and test verification for user_profile and nutrition_goal. |
| **2** | **Veer Parmar** | **26070122508** | Conceptual ER Diagram drafting (Chen notation) and Enhanced ER (EER) physical modeling. Mapped entity relationships, cardinalities, and participation constraints. Designed referential integrity rules (CASCADE vs. RESTRICT). Implemented DDL schema and nutritional catalog data for food_category and food. |

| Sr. No. | Student Name | PRN | Core Responsibilities & Modules Authored |
|---|---|---|---|
| 3 | Postiwala Vatsal Dipakbhai | 26070122509 | Database anomalies analysis on the unnormalized NUTRITION_FLAT relation. Formal Functional Dependencies formulation and mathematical normalization (1NF to BCNF). Created normalization comparison diagrams (Figures 10.1 to 10.6). Designed associative junction tables meal_item and meal_plan_item. Implemented stored functions calculate_bmi and get_total_calories, and SQL views. |
| 4 | Parth Bhayani | 26070122512 | Implemented stored procedure get_daily_summary with OUT parameters. Developed and tested database triggers (trg_after_meal_insert, trg_validate_meal_quantity). Created the system audit log module (activity_log) and recommendation table. Authored and tested the 10-query analytical test suite (Q1–Q10). Evaluated MySQL InnoDB against Codd's 12 Rules. |

## 15. Conclusion

Building NutriGuide gave our group practical experience taking a database project from an initial idea all the way to a working desktop application connected to MySQL 8.0. In class, database concepts like normalization and Boyce-Codd Normal Form can feel abstract when you are only solving written exercises on paper. But when you are actually designing tables for a meal tracker, seeing why a flat spreadsheet causes insertion and update anomalies makes the theory click immediately.

Splitting our initial design into ten normalized tables showed us why 3NF and BCNF matter in real software. If we had kept user body weights or food calorie counts in the same table as logged meals, every single portion change would have caused update anomalies. Implementing the meal_item and meal_plan_item junction tables taught us how to model many-to-many relationships cleanly without resorting to comma-separated text strings.

We also learned the importance of enforcing data integrity at the database level rather than trusting client-side forms alone. Adding CHECK constraints for positive heights, weights, and quantities ensures that bad data is rejected by MySQL InnoDB even if frontend validation is bypassed. Writing stored functions for BMI calculation and stored procedures with output parameters for daily summaries showed us how database routines can reduce repetitive client code and network traffic. Finally, using database triggers to automatically write audit logs proved how reliable backend automation can be.

Overall, this project helped us connect classroom relational algebra and SQL theory with real-world software engineering, giving us confidence in schema design, query optimization, and database application development.

## 16. References

### Textbooks & Academic Literature


1. **R. Elmasri and S. B. Navathe**, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016.
2. **A. Silberschatz, H. F. Korth, and S. Sudarshan**, *Database System Concepts*, 7th Edition, McGraw-Hill Higher Education, 2019.
3. **C. J. Date**, *An Introduction to Database Systems*, 8th Edition, Addison-Wesley, 2004.
4. **E. F. Codd**, *"A Relational Model of Data for Large Shared Data Banks"*, Communications of the ACM, Vol. 13, No. 6, 1970, pp. 377–387.
5. **E. F. Codd**, *"Is Your DBMS Really Relational?"*, Computerworld, October 1985.


### Software Documentation & Standards


6. **Oracle Corporation**, *MySQL 8.0 Reference Manual*, InnoDB Storage Engine Architecture & SQL Syntax, https://dev.mysql.com/doc/refman/8.0/en/
7. **World Health Organization (WHO)**, *Body Mass Index (BMI) Classifications & Daily Caloric Guidelines*, Geneva, Switzerland.
8. **National Institutes of Health (NIH)**, *Dietary Reference Intakes (DRIs) for Energy, Carbohydrate, Fiber, Fat, Fatty Acids, Cholesterol, Protein, and Amino Acids*.


## Appendix A. Complete SQL Script (NutriGuide.sql)

```
-- ================================================================================
-- NutriGuide Database Schema: Complete DDL & Baseline Data
-- Database Engine: MySQL 8.x | Storage Engine: InnoDB
-- ================================================================================

CREATE DATABASE IF NOT EXISTS nutriguide;
USE nutriguide;

-- Table 1: user_profile
CREATE TABLE IF NOT EXISTS user_profile (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    age INT NOT NULL CHECK (age > 0),
    gender VARCHAR(20) NOT NULL,
    height_cm DECIMAL(5, 2) NOT NULL CHECK (height_cm > 0),
    weight_kg DECIMAL(5, 2) NOT NULL CHECK (weight_kg > 0),
    activity_level VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Table 2: nutrition_goal
CREATE TABLE IF NOT EXISTS nutrition_goal (
    goal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    target_calories DECIMAL(7, 2) NOT NULL CHECK (target_calories >= 0),
    target_protein DECIMAL(6, 2) NOT NULL CHECK (target_protein >= 0),
    target_carbs DECIMAL(6, 2) NOT NULL CHECK (target_carbs >= 0),
    target_fats DECIMAL(6, 2) NOT NULL CHECK (target_fats >= 0),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_nutrition_goal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
-- Table 3: food_category
CREATE TABLE IF NOT EXISTS food_category (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
) ENGINE=InnoDB;

-- Table 4: food
CREATE TABLE IF NOT EXISTS food (
    food_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    food_name VARCHAR(100) NOT NULL UNIQUE,
    serving_size DECIMAL(6, 2) NOT NULL CHECK (serving_size > 0),
    calories DECIMAL(6, 2) NOT NULL CHECK (calories >= 0),
    protein DECIMAL(5, 2) NOT NULL CHECK (protein >= 0),
    carbohydrates DECIMAL(5, 2) NOT NULL CHECK (carbohydrates >= 0),
    fat DECIMAL(5, 2) NOT NULL CHECK (fat >= 0),
    CONSTRAINT fk_food_category FOREIGN KEY (category_id)
        REFERENCES food_category (category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 5: meal
CREATE TABLE IF NOT EXISTS meal (
    meal_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    meal_type VARCHAR(50) NOT NULL,
    meal_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 6: meal_item
CREATE TABLE IF NOT EXISTS meal_item (
    meal_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_id INT NOT NULL,
    food_id INT NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_item_meal FOREIGN KEY (meal_id)
        REFERENCES meal (meal_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 7: meal_plan
CREATE TABLE IF NOT EXISTS meal_plan (
    meal_plan_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    plan_name VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_meal_plan_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 8: meal_plan_item
CREATE TABLE IF NOT EXISTS meal_plan_item (
    meal_plan_item_id INT AUTO_INCREMENT PRIMARY KEY,
    meal_plan_id INT NOT NULL,
    food_id INT NOT NULL,
    planned_meal_type VARCHAR(50) NOT NULL,
    quantity DECIMAL(5, 2) NOT NULL CHECK (quantity > 0),
    CONSTRAINT fk_meal_plan_item_plan FOREIGN KEY (meal_plan_id)
        REFERENCES meal_plan (meal_plan_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_meal_plan_item_food FOREIGN KEY (food_id)
        REFERENCES food (food_id)
```

```
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 9: recommendation
CREATE TABLE IF NOT EXISTS recommendation (
    rec_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    rec_type VARCHAR(50) NOT NULL,
    message VARCHAR(500) NOT NULL,
    generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_recommendation_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- Table 10: activity_log
CREATE TABLE IF NOT EXISTS activity_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    action_taken VARCHAR(255) NOT NULL,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_activity_log_user FOREIGN KEY (user_id)
        REFERENCES user_profile (user_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

## Appendix B. Stored Routines & Test Suite

```
USE nutriguide;

-- Views
CREATE OR REPLACE VIEW daily_nutrition_summary AS
SELECT
    m.user_id,
    m.meal_date,
    COALESCE(SUM(f.calories * mi.quantity), 0) AS daily_calories,
    COALESCE(SUM(f.protein * mi.quantity), 0) AS daily_protein,
    COALESCE(SUM(f.carbohydrates * mi.quantity), 0) AS daily_carbs,
    COALESCE(SUM(f.fat * mi.quantity), 0) AS daily_fat
FROM meal m
LEFT JOIN meal_item mi ON m.meal_id = mi.meal_id
LEFT JOIN food f ON mi.food_id = f.food_id
GROUP BY m.user_id, m.meal_date;

CREATE OR REPLACE VIEW food_category_details AS
SELECT
    f.food_id,
    f.food_name,
    c.category_name,
    f.serving_size,
    f.calories,
    f.protein,
    f.carbohydrates,
    f.fat
FROM food f
JOIN food_category c ON f.category_id = c.category_id;

-- Stored Functions
DELIMITER $$
CREATE FUNCTION calculate_bmi(weight_kg DECIMAL(5, 2), height_cm DECIMAL(5, 2))
RETURNS DECIMAL(5, 2)
DETERMINISTIC NO SQL
BEGIN
    DECLARE height_m DECIMAL(5, 4);
    IF height_cm IS NULL OR height_cm <= 0 OR weight_kg IS NULL OR weight_kg <= 0 THEN
        RETURN 0.00;
    END IF;
    SET height_m = height_cm / 100.0000;
    RETURN ROUND(weight_kg / (height_m * height_m), 2);
END $$
```

```
CREATE FUNCTION get_total_calories(p_user_id INT, p_date DATE)
RETURNS DECIMAL(7, 2)
READS SQL DATA
BEGIN
    DECLARE total_cals DECIMAL(7, 2);
    SELECT COALESCE(SUM(f.calories * mi.quantity), 0.00)
    INTO total_cals
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
    RETURN total_cals;
END $$

-- Stored Procedure
CREATE PROCEDURE get_daily_summary(
    IN p_user_id INT,
    IN p_date DATE,
    OUT p_calories DECIMAL(7, 2),
    OUT p_protein DECIMAL(6, 2),
    OUT p_carbs DECIMAL(6, 2),
    OUT p_fat DECIMAL(6, 2)
)
BEGIN
    SELECT
        COALESCE(SUM(f.calories * mi.quantity), 0.00),
        COALESCE(SUM(f.protein * mi.quantity), 0.00),
        COALESCE(SUM(f.carbohydrates * mi.quantity), 0.00),
        COALESCE(SUM(f.fat * mi.quantity), 0.00)
    INTO p_calories, p_protein, p_carbs, p_fat
    FROM meal m
    JOIN meal_item mi ON m.meal_id = mi.meal_id
    JOIN food f ON mi.food_id = f.food_id
    WHERE m.user_id = p_user_id AND m.meal_date = p_date;
END $$

-- Triggers
CREATE TRIGGER trg_after_meal_insert
AFTER INSERT ON meal
FOR EACH ROW
BEGIN
    INSERT INTO activity_log (user_id, action_taken, log_time)
    VALUES (
        NEW.user_id,
        CONCAT('Logged meal (ID: ', NEW.meal_id, ', Type: ', NEW.meal_type, ', Date: ', NEW.meal_date, ')'),
        NOW()
    );
END $$

CREATE TRIGGER trg_validate_meal_quantity
BEFORE INSERT ON meal_item
FOR EACH ROW
BEGIN
    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Meal item quantity must be strictly greater than 0';
    END IF;
END $$
DELIMITER ;
```